



UNIVERSIDAD
POLITÉCNICA
DE MADRID

Vision Transformers

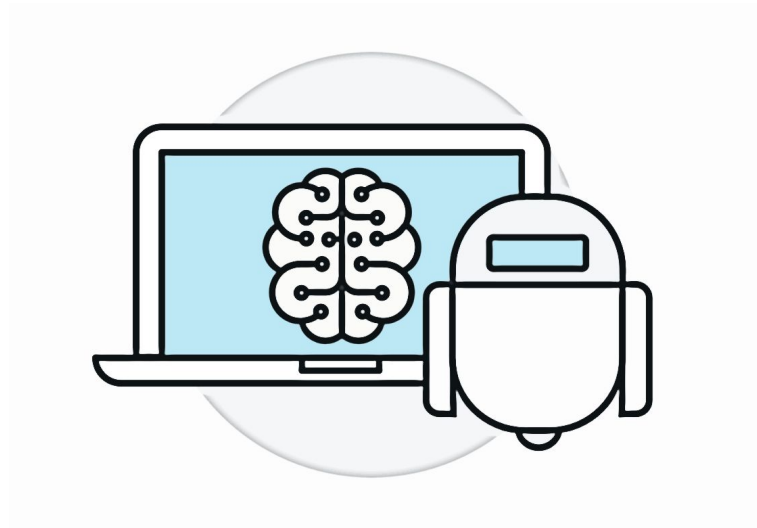
Álvaro García Barragán

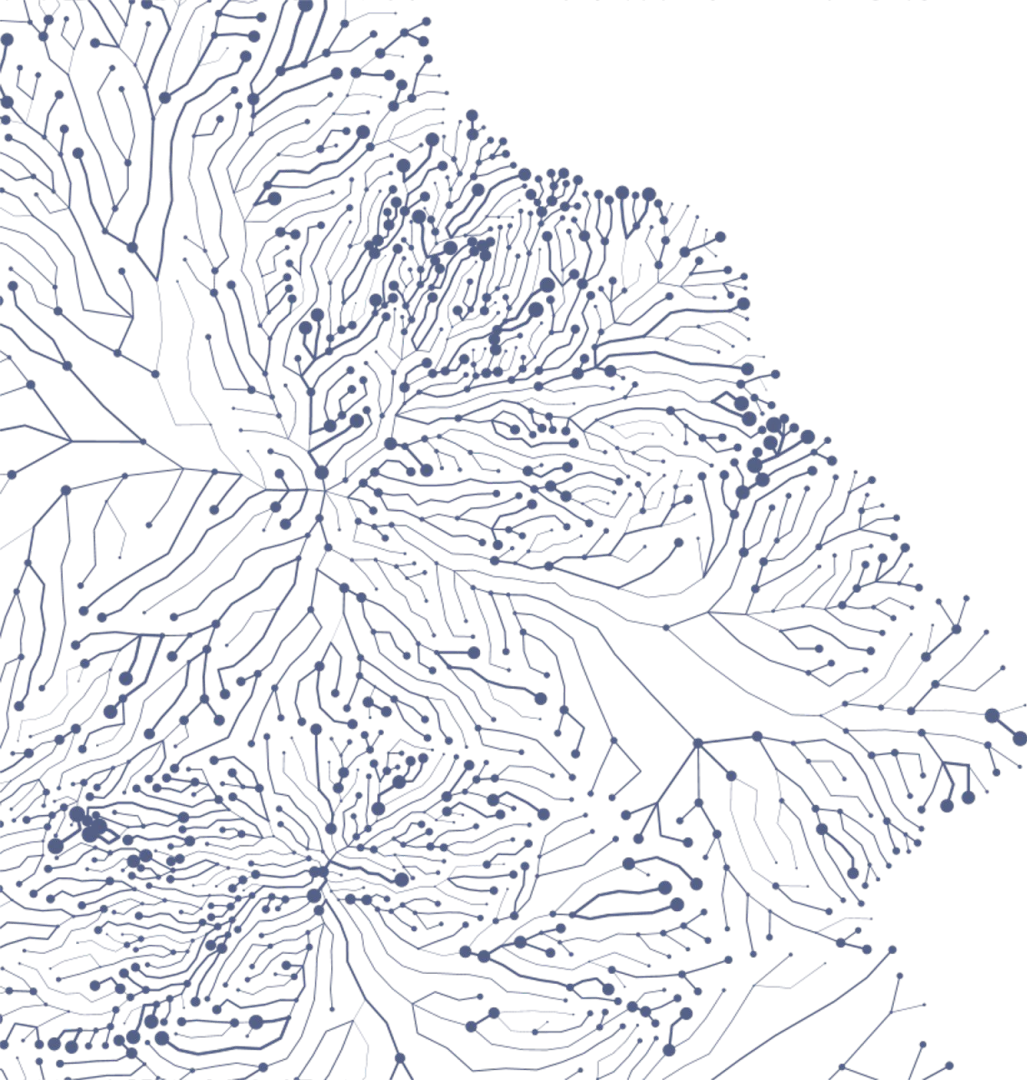
*Center for Biomedical Technology, Technical
University, Madrid*

2th April, 2024



1. Introduction
2. Natural Language Processing
3. Transformers
4. Vision Transformer
5. Conclusions

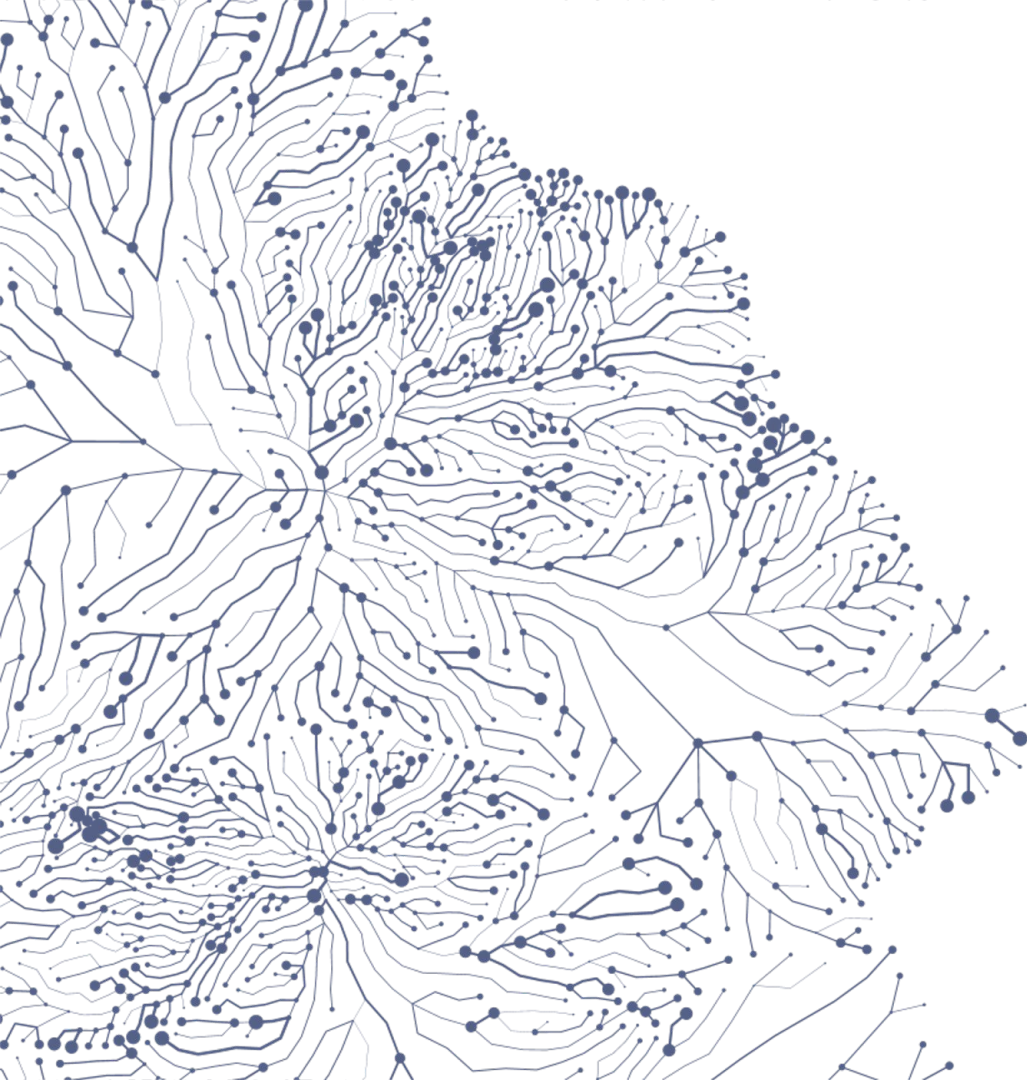




Introduction

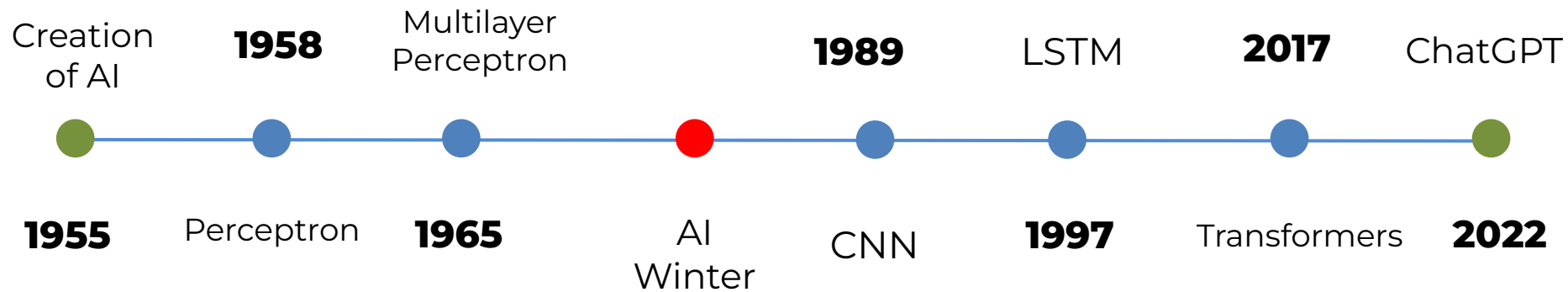
- *Can Transformers networks be used for Computer Vision problems?*
 - The success of transformers make researches to start replacing certain components of convolutional networks with self-attention (Wang et al., 2018; Carion et al., 2020)
 - Finally in 2021: "An image is worth 16 x 16 words: transformers for image recognition at scale" from Google Research. Demonstrating Transformers for independent Computer Vision Tasks and more efficient and have better results.

- The main difference in computer vision problem and natural language problem is the encoding of the input (text or image).
 - The pixels are numbers already.
 - The text must be converted to number in a way they represent the language.
 - For text is necessary encode and decode, $\text{text} \Rightarrow \text{numbers} \Rightarrow \text{text}$.
- A pixel itself have meaning? A word itself have meaning?
 - A word alone have "more" meaning than a pixel alone.

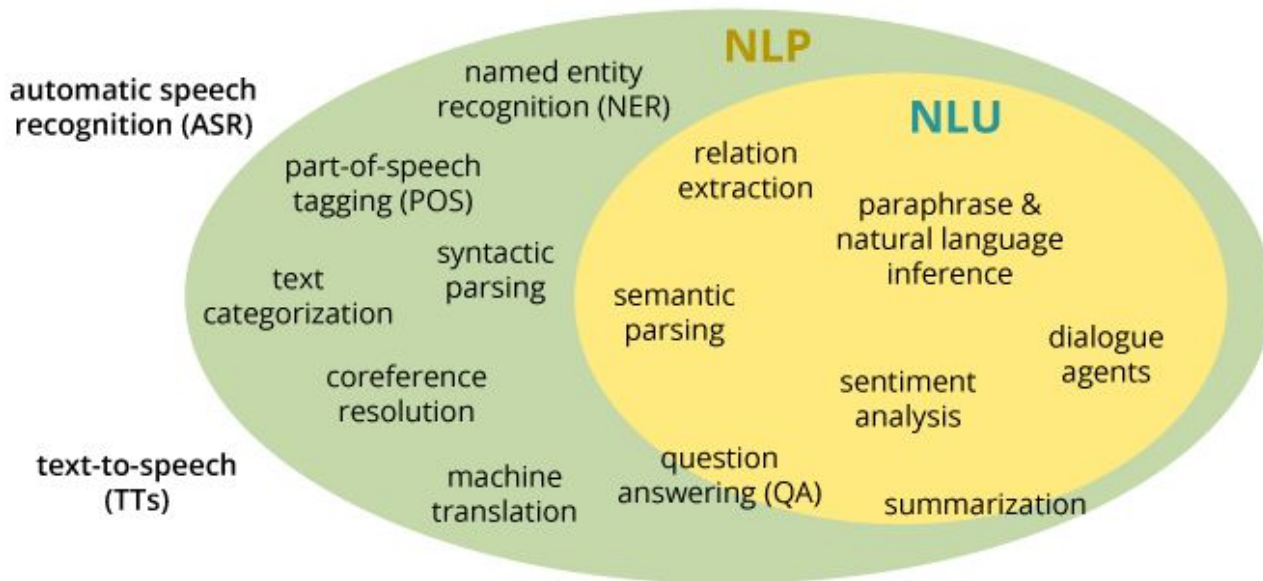
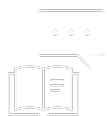


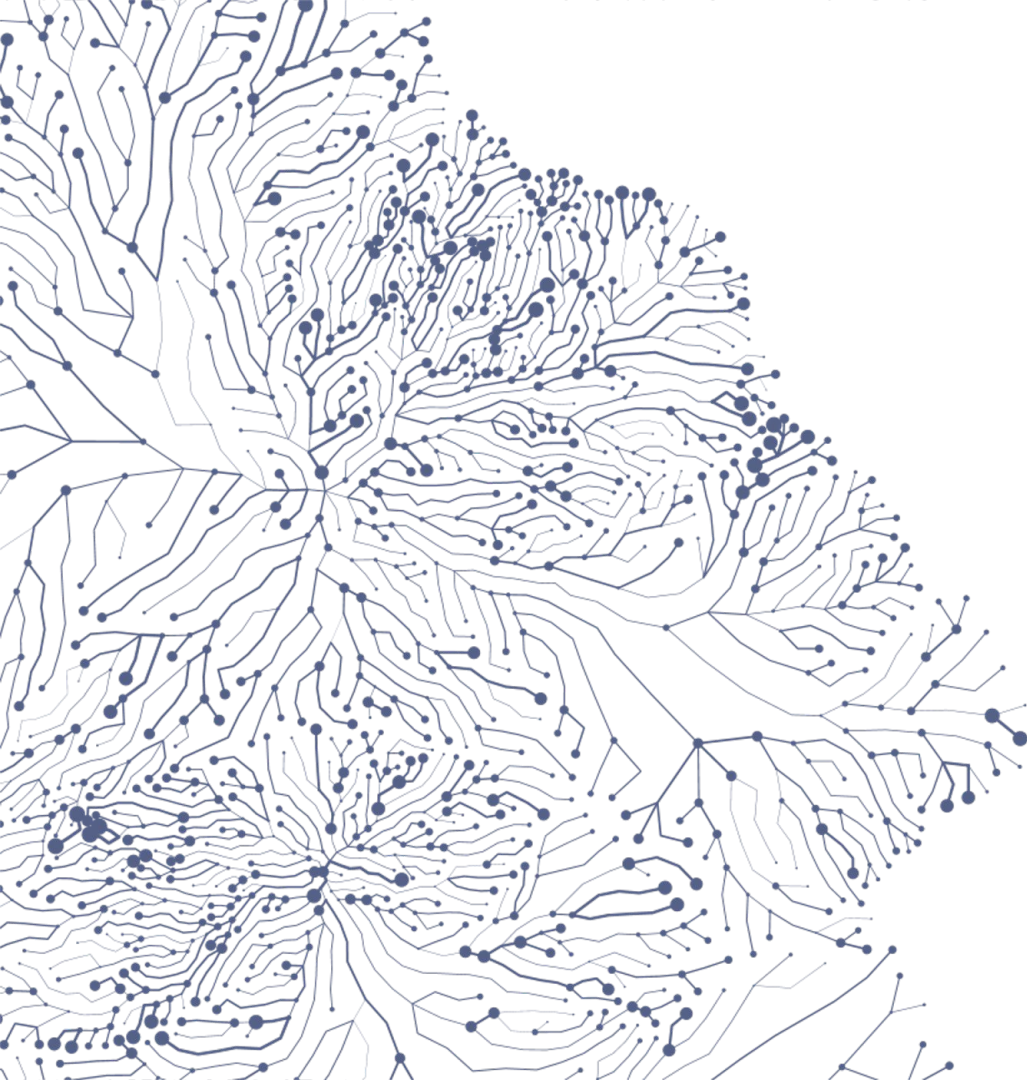
Natural Language Processing

NLP with ANN



Natural language processing (NLP) refers to the branch of AI concerned with giving computers the ability to understand text and spoken words in much the same way human beings can. [\[LINK\]](#)





NLP Basis

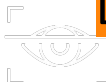
Tokenization

Computers don't
understand words only
numbers



How can you get words into a Neural Network?
Tokenization + Embeddings

 **Token** - Minimum Encoding Unit

 División por palabras

 Divi sión por sub pa la bras

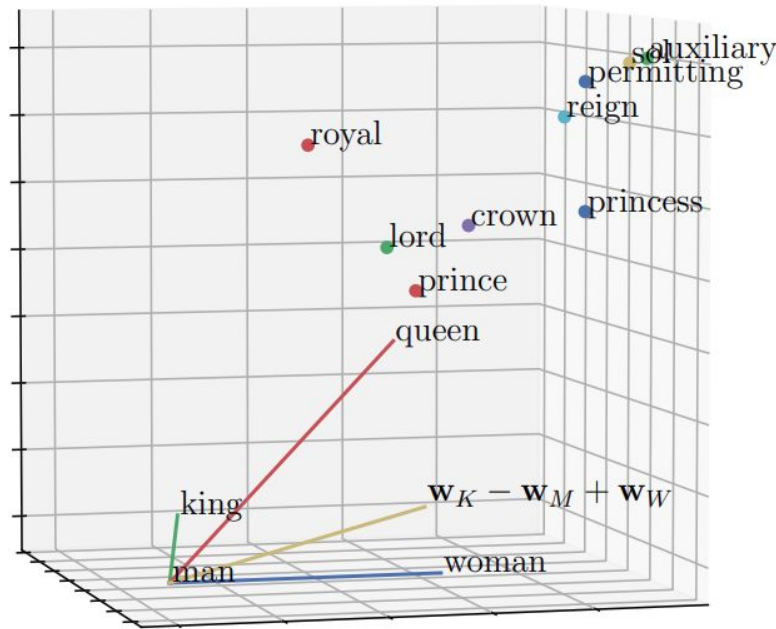
 División por caracteres

- BTE (Byte pair encoding)
(GPT, LLama)
 - In : Aaabdaaabac
 - Out : XdXac
 - $Y=ab$
 - $Z=aa$
 - $X=ZY$

Word Embeddings

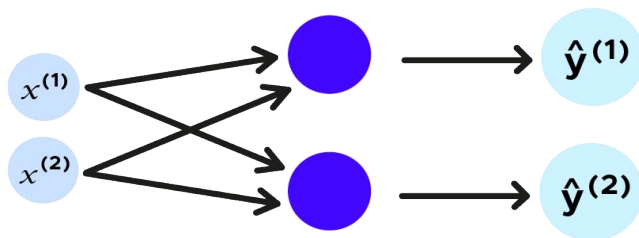


- **[Google 2013]** -> word2vec (trained model that encodes tokens to vectors). Vector Space Operation. Similar words have proximate vectors in space.
- Dimension of embeddings:
 - Word2vec: 300
 - BERT-base: 768
 - BERT-large: 1024



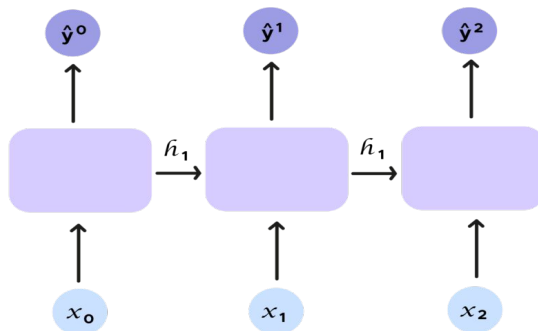
Multilayer vs RNN

MLP



Each token is independent

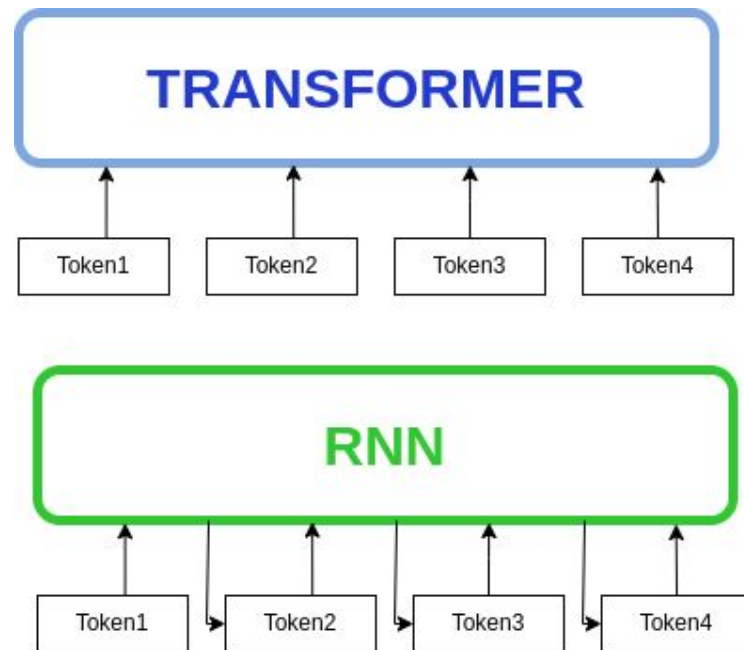
RNN

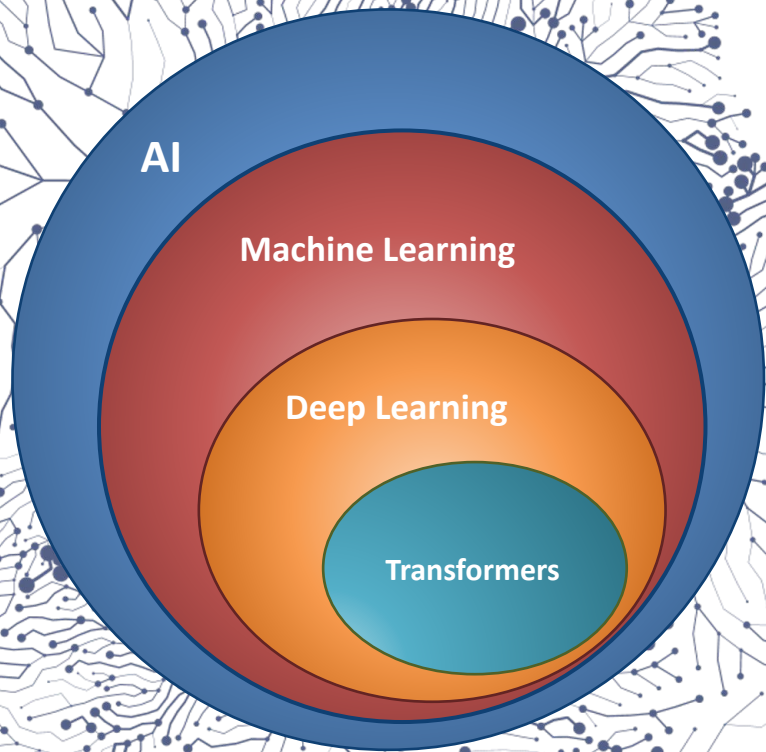


Each token depends on the previous ones

Transformers vs RNN

- In RNN, each word must wait until the previous words have been processed. RNNs have a problem with forgetfulness (gradient descent).
- On the other hand, transformers process the sentence at once and therefore parallelize the process, being able to process a greater amount of data and being more efficient in training.
- How is it achieved? **Positional Encoding**





Transformers

“Attention is all you need”

- *Attention is all you need* [2017]: Attention is a key mechanism that allows the model to focus on relevant parts of the input and produce a contextualized output (simulating human concentration).
- A new simple network architecture.
 - Based solely on attention mechanisms.
 - Dispensing of recurrence and convolutions entirely.

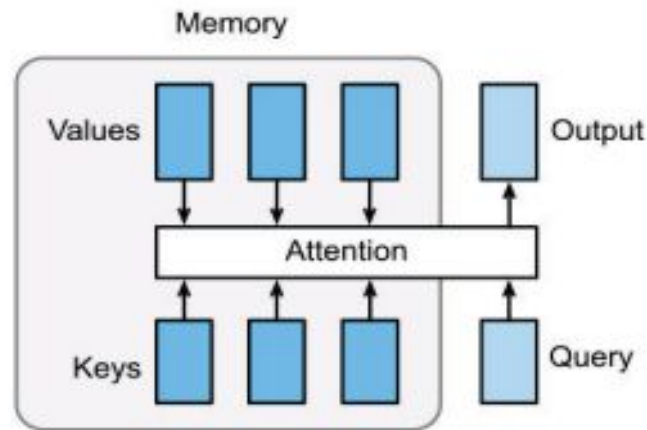
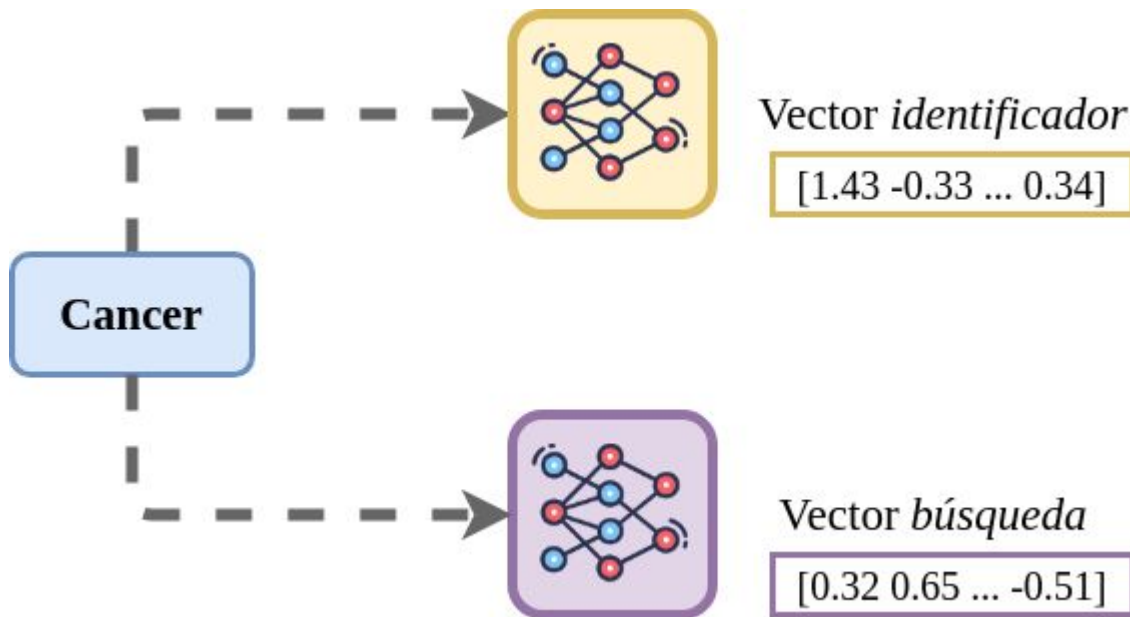


Figura 2.10: (Zhang et al., 2020a)

Self-Attention



- **Key vector:** Used to determine which input positions are important for a given position.
- **Query vector:** It is used to determine what information is relevant to the current position.

Self-Attention

- The network learns the relationship of all words with all words (*word = token*). There are words with more contextual relationship such as cancer and breast.
- The network learn that given the word “cancer” it must attend to the word “mama”



Vector Query (Lock)
Vector Key (Key)

Attention = $V_Query \times V_Key$ (Scalar Product)

Math: Self-Attention

- The attention mechanism performs the memory function.
- $R = X$ = Matrix-coded embeddings and its position.

$$X \times W^Q = Q$$

$$X \times W^K = K \quad \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

$$X \times W^V = V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

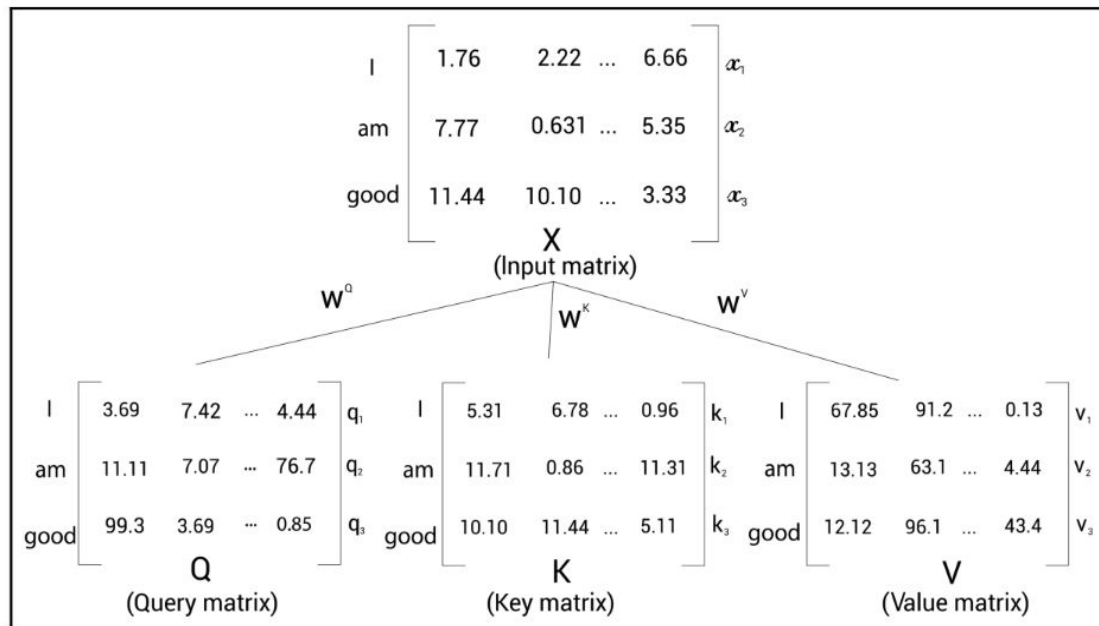
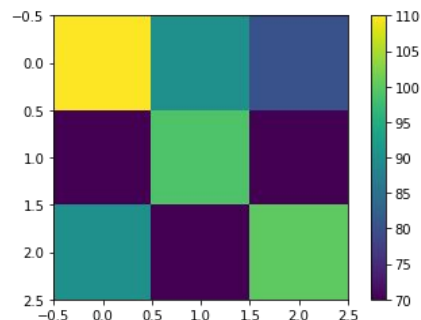
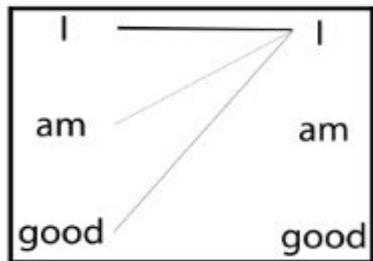


Figure 1.7 – Creating query, key, and value matrices

Math: Self-Attention



Attention Matrix = $Q \times K$

$$\begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix} \begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix} \cdot \begin{array}{c} \text{I} \quad \text{am} \quad \text{good} \\ \begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ \vdots & \vdots & \vdots \\ 0.96 & 11.31 & 5.11 \end{bmatrix} \\ \begin{matrix} k_1 \\ k_2 \\ k_3 \end{matrix} \end{array} \begin{matrix} K^T \end{matrix} = \begin{array}{c} \text{I} \quad \text{am} \quad \text{good} \\ \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 & q_1 \cdot k_3 \\ q_2 \cdot k_1 & q_2 \cdot k_2 & q_2 \cdot k_3 \\ q_3 \cdot k_1 & q_3 \cdot k_2 & q_3 \cdot k_3 \end{bmatrix} \end{array}$$

$$QK^T = \begin{array}{c} \text{I} \quad \text{am} \quad \text{good} \\ \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix} \end{array}$$

Note: The value matrix stores learned memories, so when provided with a query and a key (representing how words are interconnected), it recalls this information from the past.

Math: Positional Encoding



- The most important key concept.
- Representing the position with cosine and sine in a way that does not disturb the embedding.

$$P(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right)$$

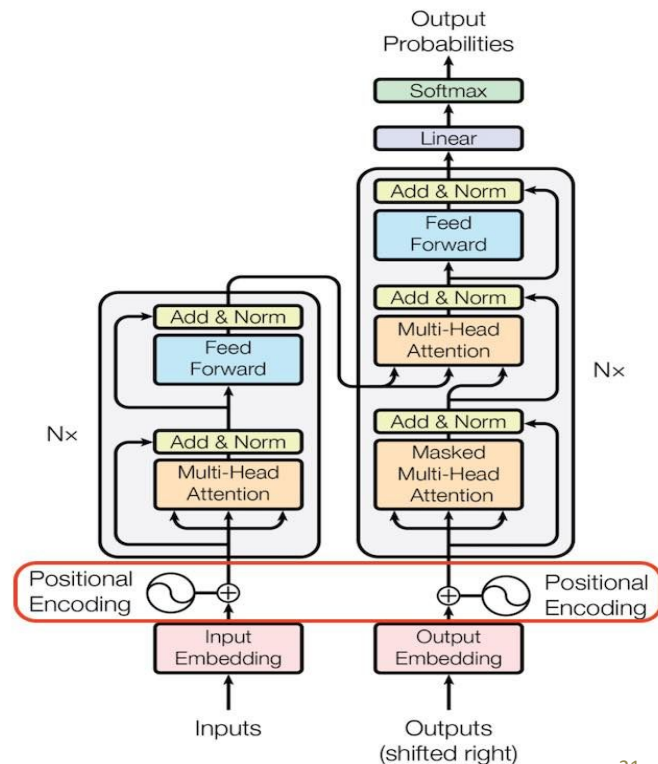
$$P(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right)$$

		$\sin(0)$	$\cos(0)$	$\sin(0/100)$	$\cos(0/100)$
P =	am	$\sin(1)$	$\cos(1)$	$\sin(1/100)$	$\cos(1/100)$
	good	$\sin(2)$	$\cos(2)$	$\sin(2/100)$	$\cos(2/100)$

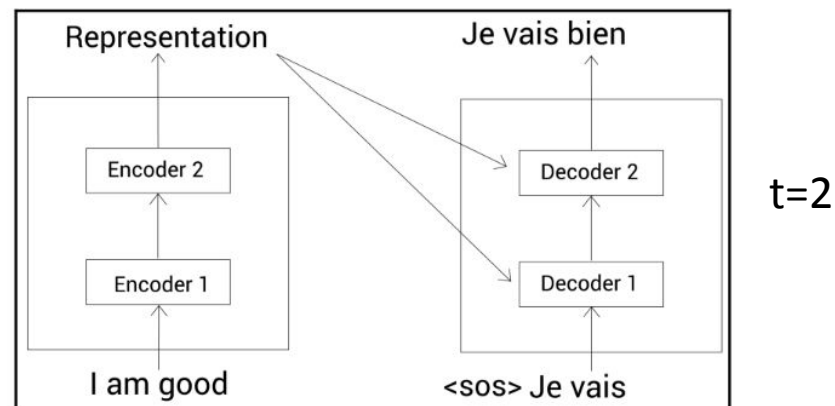
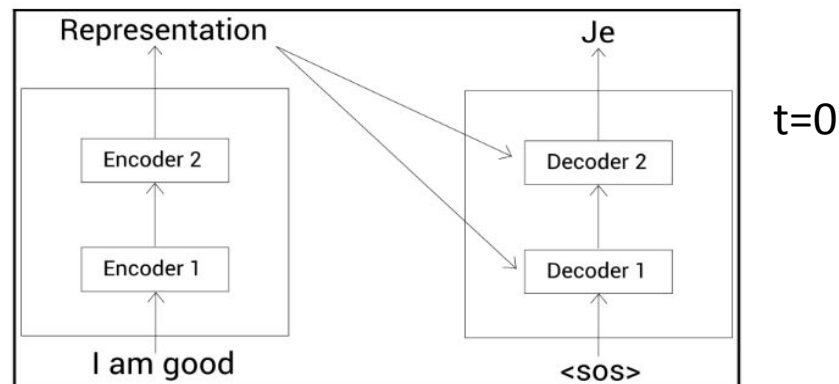
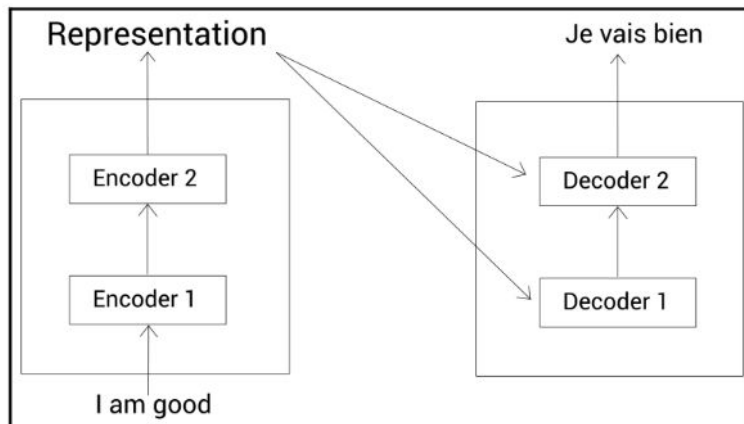
		$\sin\left(\frac{\text{pos}}{1000^0}\right)$	$\cos\left(\frac{\text{pos}}{1000^0}\right)$	$\sin\left(\frac{\text{pos}}{1000^{2/4}}\right)$	$\cos\left(\frac{\text{pos}}{1000^{2/4}}\right)$
P =	am	$\sin\left(\frac{\text{pos}}{1000^0}\right)$	$\cos\left(\frac{\text{pos}}{1000^0}\right)$	$\sin\left(\frac{\text{pos}}{1000^{2/4}}\right)$	$\cos\left(\frac{\text{pos}}{1000^{2/4}}\right)$
	good	$\sin\left(\frac{\text{pos}}{1000^0}\right)$	$\cos\left(\frac{\text{pos}}{1000^0}\right)$	$\sin\left(\frac{\text{pos}}{1000^{2/4}}\right)$	$\cos\left(\frac{\text{pos}}{1000^{2/4}}\right)$

Transformers: *key concepts*

- New architecture (2017) that allows for the **parallel processing** of data sequences (pixels from images, words from sentences, or sequences of amino acids).
- Input Embedding still maintains the same functionality of converting words into numerical vectors.
- Residual connections that prevent the problem of gradient descent (loss of memory).
- First proposal for translation: $x = 6$, meaning 6 encoders and 6 decoders.
- Two main concepts: **Self-Attention** and **Positional Encoding**.



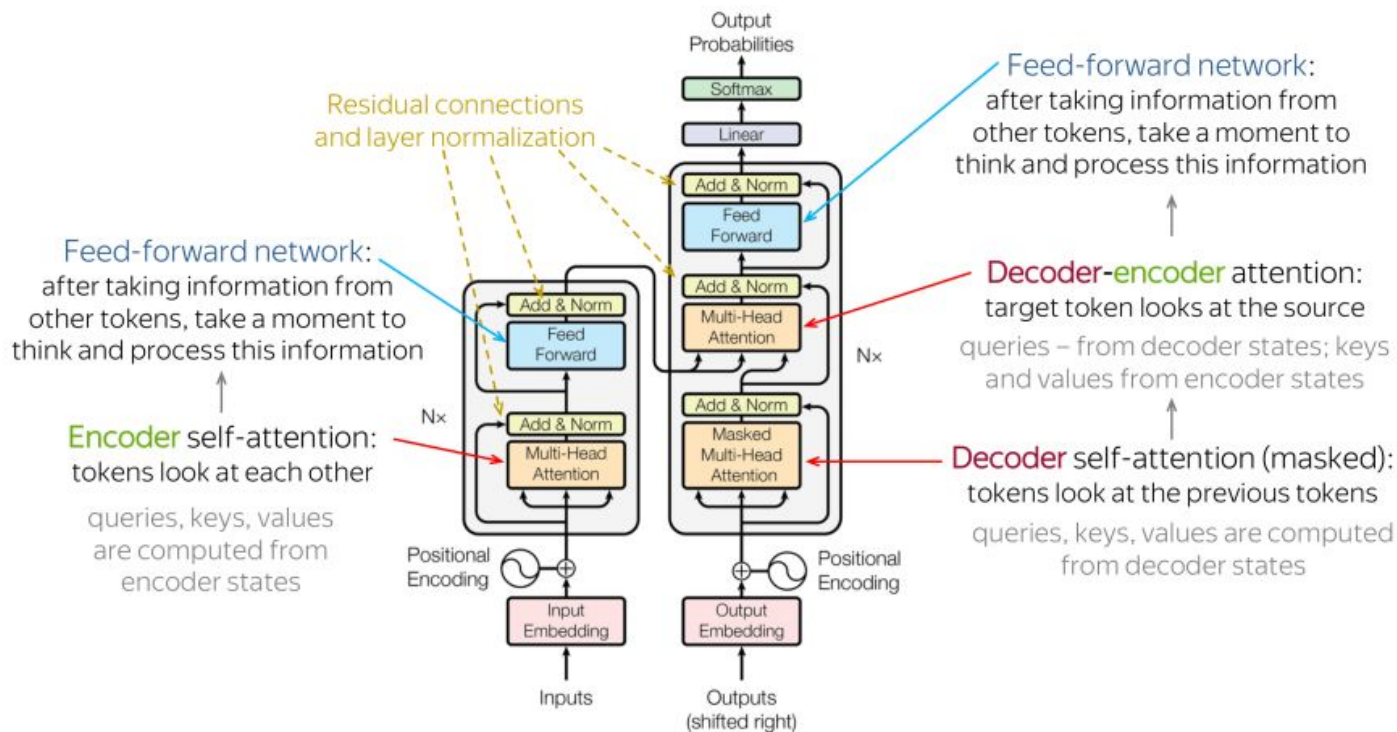
Translate example

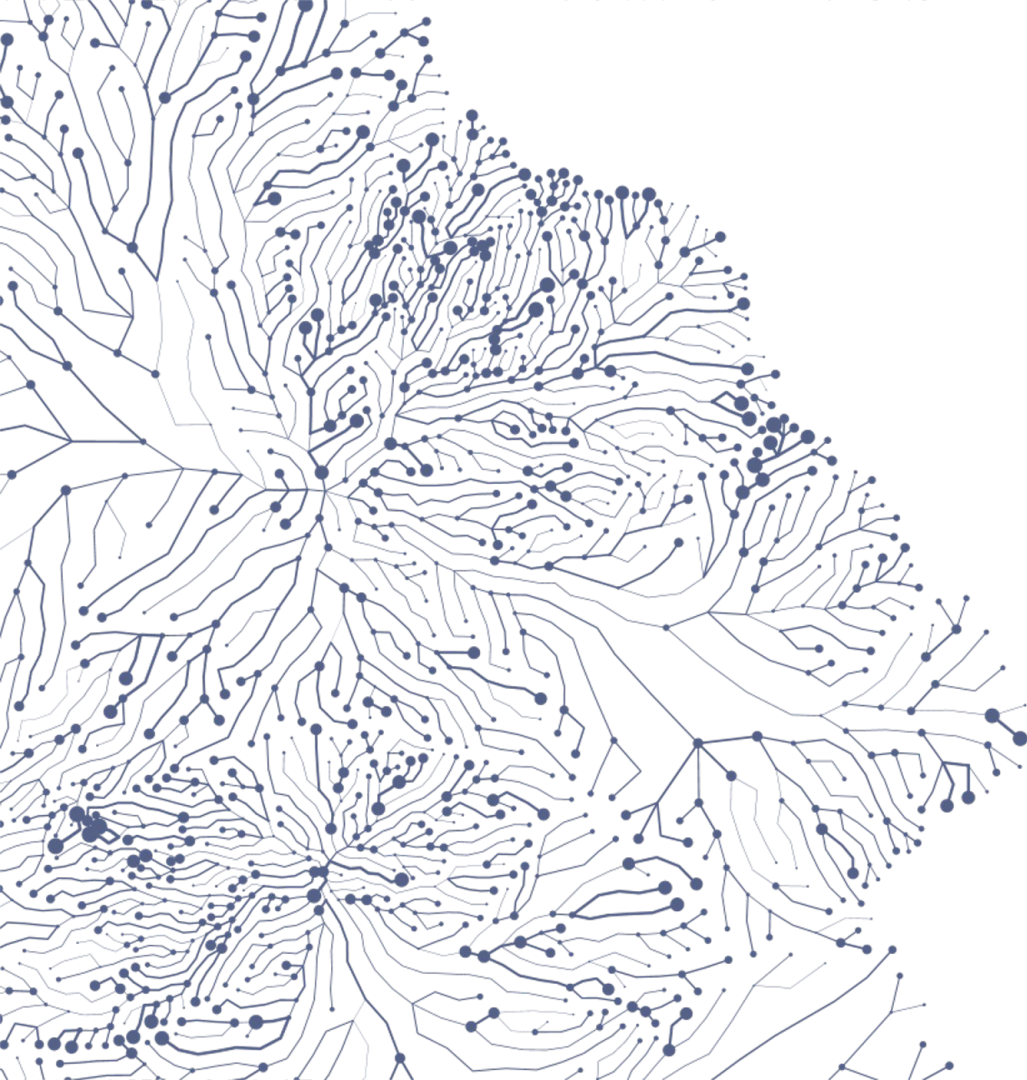


 medal

- [illegible]

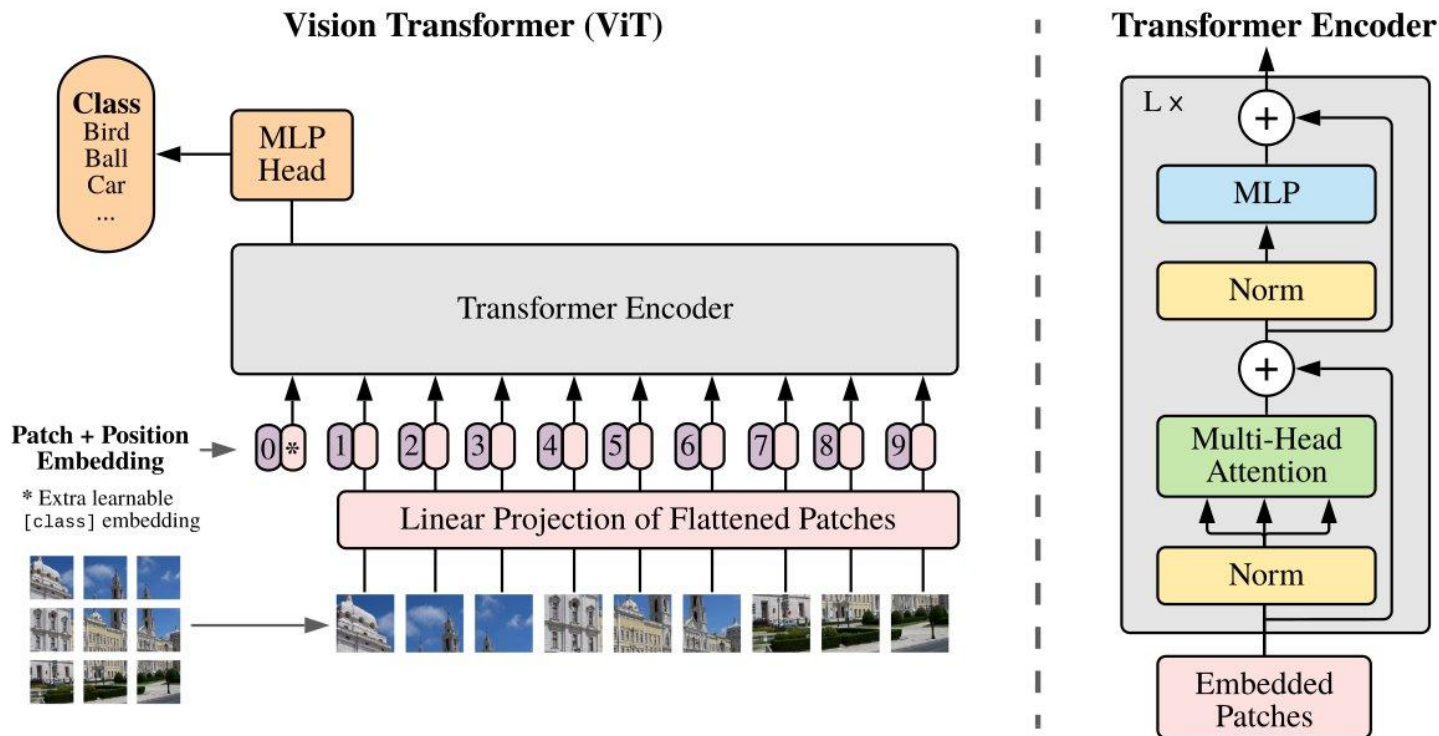
Transformers Architecture





ViT - Vision Transformers

ViT Architecture: *Encoder-Only*

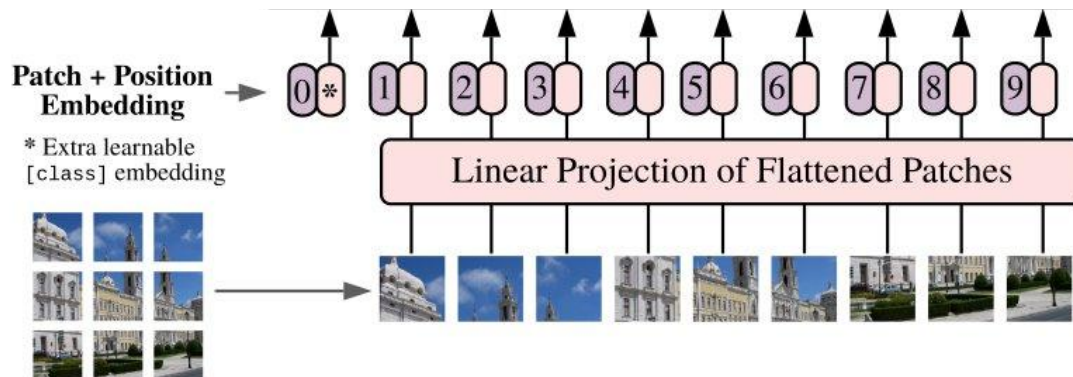


The images are passed to the network as a text

ViT Architecture: *Preprocessing* (*img2words2vecs*)



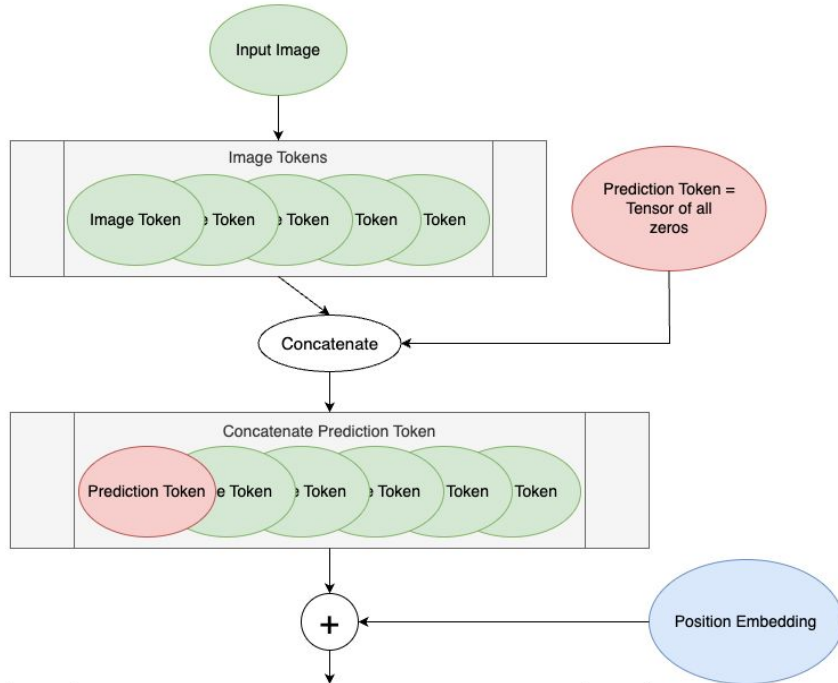
1. The image is first divided into patches.
2. Each patch is then flattened into a one-dimensional vector. ($P^2 * C = \text{length before projecting}$)
3. These vectors are then passed through a trainable linear projection layer, which reduces their dimensionality to D (*size path embeddings = latent dimension = channel dimension = token length*).
4. Position embeddings are added to the patch embeddings to retain positional information.
5. The resulting lower-dimensional vectors (embeddings) are then fed into the transformer encoder.



P = Path size
C = Number of Channels
D = Token Length

P=16, H=224, W=224
D = $16^2 \times 3 = 768$
N = $224 \times 224 / 16 = 3136$

ViT: Step-by-step



1. **Tokenization:** the ViT converts an image to tokens such that each token represents a local area (*patch*) of the image. Images of height H , width W , and channels C into N tokens with patch size P .

- $N = H * W / P^2$
- Each token is of length $D = P^2 * C$.

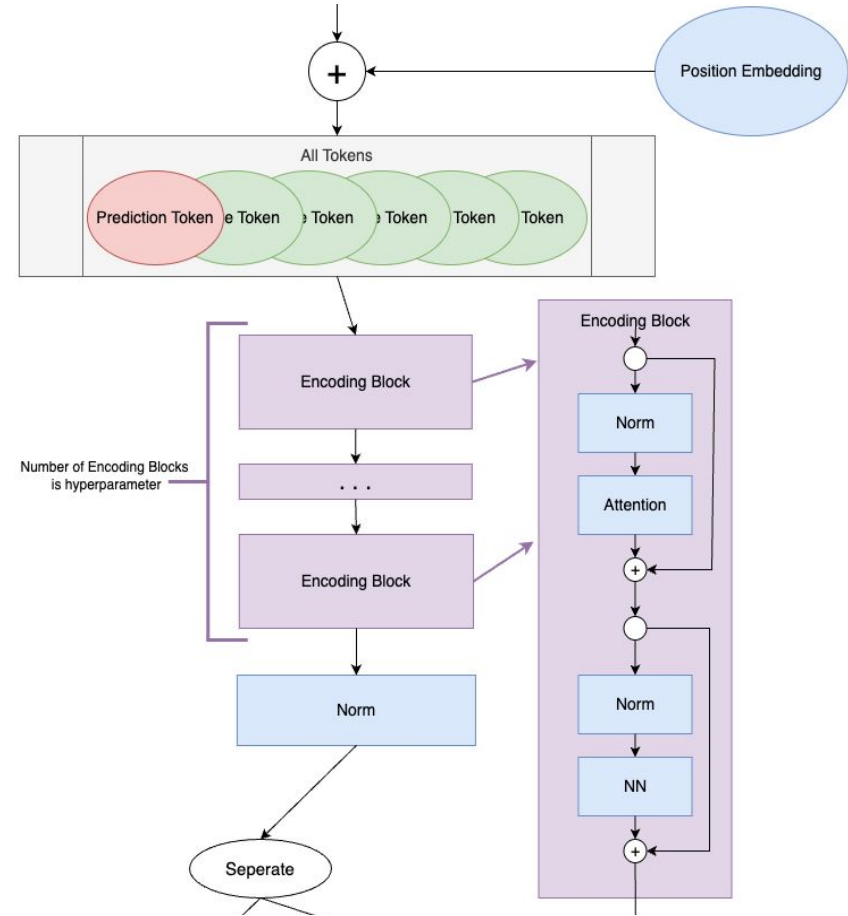
2. **Concatenate Prediction Token:** This token will be used at the output of the encoding blocks to make a prediction. It starts off blank so that it can gain information from the other image tokens.

Note: The Transformer's sequence length is inversely proportional to the square of the patch size, thus models with smaller patch size are computationally more expensive. (this example don't have the linear projection step)

ViT: Step-by-step

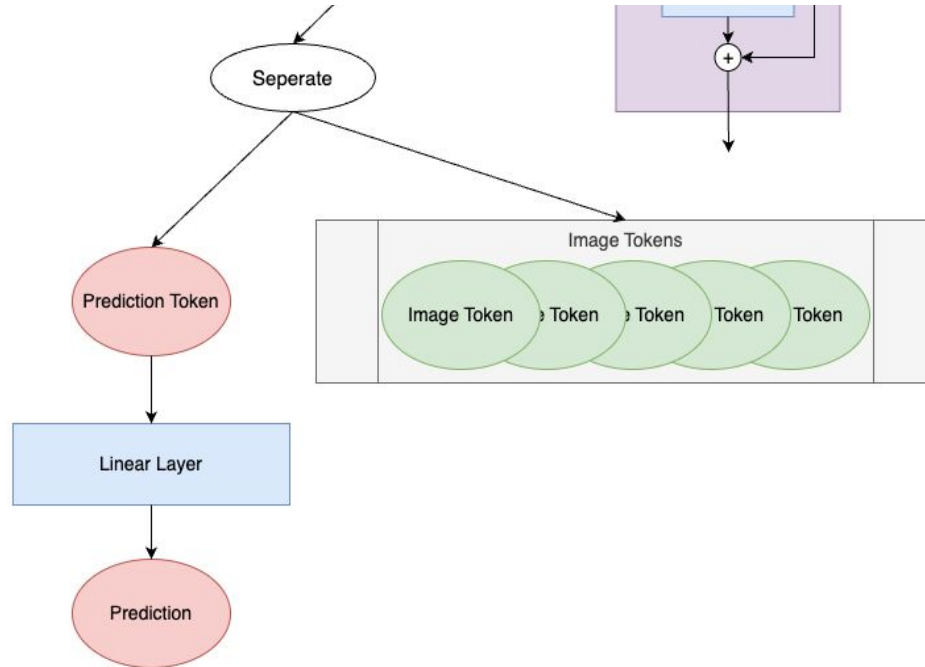
3. Adding of Position Embedding: Position embeddings are added to the patch embeddings to retain positional information. (1D position embeddings, 2D not significant performance). The position embedding allows the transformer to understand the order of the image token. (like transformer for text)

4. Encoder Blocks: The processed token pass to the encoding block is where the model learns from the image tokens. The number of encoding blocks is a hyperparameter set by the user.



ViT: Step-by-step

5. Prediction Processing: After passing through the encoding blocks, the prediction tokens are separated and are passed to the Linear Layer component that make a prediction.



ViT Attention

- Image tokens after passing from attention layers. (Attention Rollout)
- Model attends to image regions that are semantically relevant for classification.

Input



Attention



ViT Foundation Models

BERT-base = 110M
GPT3 = 175B



- Pretrained models trained for days.
- The “Base” and “Large” models are directly adopted from BERT and add the larger “Huge” model.
- **Notation:** for instance, ViT-L/16 means the “Large” variant with 16X16 input patch size.

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Layers: number of transformer encoder blocks.

MLP size: dimensionality of the fully-connected layer in encoder blocks.

Heads: number of attention heads within the multi-head self-attention layer.

Finetuning for Image Classification: Benchmarks



- All models were trained on Google TPUv3 hardware.
- **Computational power measure:** Number of TPUv3-core-days taken to pre-train (total processing power available, not a specific amount of time)
- “JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less computational resources to pre-train”.

SOTA Models

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Conclusions



- Vision transformers is the **state of art in performance**:
 - More efficient than SOTA models.
 - Higher accuracy.
- Pretraining is an expensive computational task but is very useful as then we can finetune in another downstream task.
 - **Transfer learning**: pretraining during hours to days and finetuning minutes to hours.
 - The models learn how to represent the language/images.
- Open the possibility of combine NLP world and Computer Vision => Multimodal models.
- “a picture is worth a thousand words” => “An image is worth 16x16 words”
- The end of CNNs?

Bibliography



- Dosovitskiy, Alexey, et al. "An image is worth 16x16 words: Transformers for image recognition at scale." arXiv preprint arXiv:2010.11929 (2020).
- *Vision Transformers, Explained:*
<https://towardsdatascience.com/position-embeddings-for-vision-transformers-explained-a6f9add341d5>
- *Position Embeddings for Vision Transformers, Explained:*
<https://medium.com/towards-data-science/vision-transformers-explained-a9d07147e4c8>
- Attention is all you need, A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, {, Kaiser, and I. Polosukhin. Advances in Neural Information Processing Systems , page 5998--6008. (2017)
- Getting Started with Google BERT: Build and train state-of-the-art natural language processing models using BERT.
- Transformers for Natural Language Processing: Build, train, and fine-tune deep neural network architectures for NLP with Python, PyTorch, TensorFlow, BERT, and GPT-3.
- García-Barragán, Alvaro, et al. "Structuring Breast Cancer Spanish Electronic Health Records Using Deep Learning." 2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS). IEEE, 2023.
- DotCSV youtube channel, Artificial Intelligence science communicator.

Thanks!

Álvaro García Barragán

alvaro.gbarragan@alumnos.upm.es

**Escuela Técnica Superior de Ingenieros Informáticos,
Universidad Politécnica, Madrid**

